

GSoC21 W10: LFortran Runtime Library Design

Rohit Goswami · 2021-08-14 · gsoc21 · fortran · ramblings

The road to accurate elementary intrinsic functions is paved with IEEE/ISO standards

Background

Serialized update for the [2021 Google Summer of Code](#) under the [fortran-lang organization](#), mentored by [Ondrej Certik](#).

Logistics

- Met with Ondrej daily from Sunday till Friday
 - On Sunday we finished the compile time evaluation design (partially)
 - Ondrej finished the rest overnight
 - On Monday, we started co working on runtime implemenations
 - Also added some more compile time stuff and pointers to clean things up
 - Ondrej fixed a longstanding bug in the deserialise of serialized mod
 - On Wednesday there was more scaffolding work done on the generics
 - Added another implicit cast
 - On Thursday we implemented an all fortran `sin` function
 - Just as accurate as `gfortran`
 - Compiles with LFortran
 - Also fixed another serialization bug (floats were being truncated)
 - On Friday we redesigned the runtime library
 - Also discussed future plans and longterm goals
 - Opened discussions around the runtime library and elementary function accuracy on the Zulip instance
 - Considered adding `fortran` versions of LAMMPS (F77 and F90) to the compilation plans

Overview

This was most definitely my most productive week by far.

New Merge Requests

[llvm: More values when possible \(1113\)](#)

Backend values from compile time evaluations

[kind: Compile time \(1132\)](#)

A rather thorny refactor; but one with great benefits

asr: Clean up extract_kind (1136)

Fruitful usage of the new functions

Compile Time Intrinsic: Real (1114)

A `cast` based approach

selected_kinds: Body visitors (1167)

Possibly my favorite MR, a nice pure `fortran` implementation backed by a rough `sollya` approximation

Initial sin implementation (1167)

Possibly my favorite MR, a nice pure `fortran` implementation backed by a rough `sollya` approximation

sin: Rework runtime to get better approximations (1173)

Possibly my favorite MR, a nice pure `fortran` implementation backed by a rough `sollya` approximation

Compile time floor (1176)

Fairly standard implementation requiring casts

Integer2real (1162)

An implicit cast which had not previously been implement

Compile time int (1169)

Along the lines of `real()`

Sincos tests (1194)

Benchmarking and setting up of the new test framework

Freshly Assigned Issues

- [Semantics: Implement correct handling of PARAMETER \(510\)](#)
- [ASR Pass: Order functions \(507\)](#)
- [Semantics: Implement FLOOR \(A \[, KIND\]\) \(506\)](#)
- [Semantics: Implement MODULO \(A, \[, KIND\]\) \(504\)](#)
- [\[RFC\] Compile Time Intrinsic Design \(501\)](#)
- [Strange Runtime issue with Implicit Casts \(500\)](#)
- [Implicit cast for parameter \(499\)](#)
- [Better handling of intrinsic module names \(494\)](#)

Additional Tasks

Will write a closing report for next week.

Misc Logs

This was a really packed week. Much of the discussion has already taken place in the relevant issues and MRs; but some notes are in order for posterity.

Items to fix

- Some compile time function implementations are not standard compliatn
 - `real`

- Implicit casting rules fail for certain cases
 - Specifically for all `Arrays`

Strange bug when intrinsic functions are called on implicit cast

Generic Procedures

This was fun to work on.

Fortran Runtime Intrinsics

A long term goal is to have a dog-footed runtime library. We zeroed in on this design over on the wiki a few weeks ago. Now with generic procedures in; we've begun implementing some of the intrinsic functions.

Simple Functions

The simplest ones are of course trivial:

```
abs
```

Trigonometric functions

For more “mathematical” methods, we needed a template. One we found which was compatible with our license is the `fdlibm` versions. Consider for `sin`:

- We have `s_sin` and also `k_sin`
 - Written in C and assembly

Older compilers are much more readable compared to the newer ones.

Trigonometric Functions

The quote is motivated by my (limited) reading of two texts in the field of implementing elementary functions, namely Muller (2016) and Muller et al. (2018).

Brainstorming Sin

- [This](#) and then the [simpler version here](#)
- Multiple interval based approximation used for [AMD's optimized sin](#)

We were looking for a pedagogical, simple approach to calculating `sin`. Unsurprisingly, the entire ecosystem of compilers seems to mostly follow one implementation from the early 90s. Which is great, but Fortran doesn't really implement IEEE 754's argument reduction routine (a.k.a. `quadmath`). It turns out with a little massaging; the pedagogical [approach presented in this blog post by Christer Ericson](#) can be adapted for use.

We needed to do slightly better. For starters, the errors of around 10^{-16} could be stomached, but not the ones around 10^{-10} . Think of this as the same sort of problem as the standard fitting of a potential energy surface.

How do we test these? Import from the intrinsic, and rename to a different symbol, then run a loop.

Sollya and Function Approximations

Rather than go rooting around the annals of outdated compilers; and instead of boldly copying from existing libraries; an interesting approach is to calculate precise approximations including the Remez (or minmax), Taylor approximations, and Chebyshev approximations.

Thankfully testing this is trivial with Sollya. Since it does not have a terminal inbuild we need `rlwrap` too.

```
nix-shell -p sollya rlwrap
```

Consider a trial implementation:

```
! lfortran_intrinsic_sin.f90
module lfortran_intrinsic_sin
use, intrinsic :: iso_fortran_env, only: sp => real32, dp => real64
implicit none

interface sin
  module procedure dsin
end interface

contains

elemental real(dp) function dsin(x) result(r)
real(dp), parameter :: pi = 3.1415926535897932384626433832795_dp
real(dp), intent(in) :: x
real(dp) :: y
integer :: n
if (abs(x) < pi/2) then
  r = kernel_dsin(x)
else ! fold to pi/2
! https://realtimecollisiondetection.net/blog/?p=9
  y = modulo(x, 2*pi)
  y = min(y, pi - y)
  y = max(y, -pi - y)
  y = min(y, pi - y)
  r = kernel_dsin(y)
end if
end function
! Implement kernel_dsin with Sollya (read post)
```

The general approach to testing this is then:

```
! test_sina.f90
program main
  use :: lfortran_intrinsic_sin, only: sin2 => dsin
  real(kind=8) :: x,y
  integer(kind=8) :: i
  do i=1,1000000
    y = real(i)/100.0
    print*, y, abs(sin(y)-sin2(y))
  end do
end program
```

Which can be compiled with:

```
gfortran -ffree-line-length-none lfortran_intrinsic_sin.f90 test_sina.f90 && ./a.out
```

Where the line length is important since we want to be able to copy and paste expressions from Sollya.

Remez

The test expressions can be generated as follows:

```
nix-shell -p sollya rlwrap
rlwrap -A sollya
prec=120;
f=sin(x);
d=[-pi/2;pi/2];
p=remez(f,16,d,default,1e-16,1e-30);
p;
1.430896392112983438690712915149618875e-17 + x * (0.9999999999999989714281994951487325976 + x *
(-3.349530544567829511297283045847078806e-16 + x * (-0.16666666666666470131512533841331726124 + x *
(2.06038248351960358085265034956173799e-15 + x * (8.33333333322297677610158866083573121e-3 + x *
(-5.200037492550081347598442689963074914e-15 + x * (-1.984126981335937217532222096272076263e-4 + x *
(6.779661505197324337448766161086156236e-15 + x * (2.755731547177283303366066506722157076e-6 + x *
(-4.994033469644991382509162854527493863e-15 + x * (-2.505182167856013032537240066679691216e-8 + x *
(2.101039405474559290547827191709051192e-15 + x * (1.604654097883256924434495617086734883e-10 + x *
(-4.71033700061518989435986419744537652e-16 + x * (-7.356804698944816248097669067062646244e-13 + x *
4.36551332303930597866240036869066221e-17))))))))))))))
```

Where the signature of `remez` is `func, degree, interval, default=1, quality=1e-5, bounds=[0;infy]`; Now we can implement `kernel_dsin` as follows:

```
elemental real(dp) function kernel_dsin(x) result(res)
  real(dp), intent(in) :: x
  res = 1.430896392112983438690712915149618875e-17_dp + x * (0.9999999999999989714281994951487325976_dp + x *
(-3.349530544567829511297283045847078806e-16_dp + x * (-0.16666666666666470131512533841331726124_dp + x *
(2.06038248351960358085265034956173799e-15_dp + x * (8.33333333322297677610158866083573121e-3_dp + x *
(-5.200037492550081347598442689963074914e-15_dp + x * (-1.984126981335937217532222096272076263e-4_dp + x *
(6.779661505197324337448766161086156236e-15_dp + x * (2.755731547177283303366066506722157076e-6_dp + x *
(-4.994033469644991382509162854527493863e-15_dp + x * (-2.505182167856013032537240066679691216e-8_dp + x *
(2.101039405474559290547827191709051192e-15_dp + x * (1.604654097883256924434495617086734883e-10_dp + x *
(-4.71033700061518989435986419744537652e-16_dp + x * (-7.356804698944816248097669067062646244e-13_dp + x *
4.36551332303930597866240036869066221e-17_dp))))))))))))))

end function
```

Note that we need to use double precision for the numbers, so we can replace `SPACE+` with `_dpSPACE+`. We would like to also get some statistics of the output, in `bash` just because we can `[^fn:1]`.

```
gfortran -ffree-line-length-none lfortran_intrinsic_sin.f90 test_sina.f90 && ./a.out > remez16
awk 'NR == 1 || $2 < min {line = $0; min = $2}END{print line}' remez16
awk 'NR == 1 || $2 > max {line = $0; max = $2}END{print line}' remez16
```

Taylor and Chebyshev

To extract information from the “certified” functions in `Sollya` we need to remember that the return value is a list.

```
nix-shell -p sollya rlwrap
rlwrap -A sollya
prec=120;
f=sin(x);
d=[-pi/2;pi/2];
p=chebyshevform(f,16,d);
p[2];
p[0];
quit;
```

Similarly `taylorform` can be used to get Taylor series expansions. Given that we know exactly what we want to generate; we can also simply use `sollya` with an input script:

```
sollya < taylorform16
```

Where `taylorform16` contains:

```
f=sin(x);
d=[-pi/2;pi/2];
p=taylorform(f,16,d);
p[0];
```

This workflow can be easily extended into a script based runner.

Better Polynomials

Some basic understanding of series expansions and / or visual inspection would make it clear that for `sin` at-least; the odd terms are essentially 0. `Sollya` can handle these; as described in Muller (2016). We opted to go the rounding error route.

Rounding Errors

With Ondrej, we were able to isolate the inaccuracy in our pure `fortran` implementation and we realized it stemmed from the manner in which our argument reduction is carried out. This is a well known problem with attempted solutions from 1980s Payne and Hanek (1983) (including a related implementation in BSD) [^fn:2]. In particular, as per this [WG document of the SunPro group](#) Ng (2006) describing the canonical implementation; the error is actually from the division and difference. This is also touched upon elsewhere in the literature, Brisebarre et al. (2005) and Defour et al. (n.d.), along with Muller et al. (2018) . Naturally one of first places to check is the relevant [ISO draft standard on elementary functions \(N462Pdf?\)](#) .

5.3.9.2 Radian sine

The $\sin_F^*$ approximation helper function:

$$\sin_F^* : \mathcal{R} \rightarrow \mathcal{R}$$

$\sin_F^*(x)$ returns a close approximation to $\sin(x)$ in $\mathcal{R}$ if $|x| \leq \text{big_angle_}r_F$, with maximum error $\text{max_error_}\sin_F$.

Further requirements on the $\sin_F^*$ approximation helper function are:

$$\begin{aligned} \sin_F^*(n \cdot 2 \cdot \pi + \pi/6) &= 1/2 & \text{if } n \in \mathcal{Z} \text{ and } |n \cdot 2 \cdot \pi + \pi/6| \leq \text{big_angle_}r_F \\ \sin_F^*(n \cdot 2 \cdot \pi + \pi/2) &= 1 & \text{if } n \in \mathcal{Z} \text{ and } |n \cdot 2 \cdot \pi + \pi/2| \leq \text{big_angle_}r_F \\ \sin_F^*(n \cdot 2 \cdot \pi + 5 \cdot \pi/6) &= 1/2 & \text{if } n \in \mathcal{Z} \text{ and } |n \cdot 2 \cdot \pi + 5 \cdot \pi/6| \leq \text{big_angle_}r_F \\ \sin_F^*(x) &= x & \text{if } \sin_F^*(x) \neq \sin(x) \text{ and } |x| \leq \sqrt{3 \cdot \text{epsilon}_F / r_F} \\ \sin_F^*(-x) &= -\sin_F^*(x) \end{aligned}$$

The $\sin_F$ operation:

$$\begin{aligned} \sin_F : F &\rightarrow F \cup \{\text{underflow}, \text{absolute_precision_underflow}\} \\ \sin_F(x) &= \text{trans_result}_F(\sin_F^*(x), \text{nearest}_F) & \text{if } x \in F \text{ and } \text{fmin}N_F < |x| \text{ and } |x| \leq \text{big_angle_}r_F \\ &= \text{rad}_F(x) & \text{otherwise} \end{aligned}$$

NOTE – **underflow** is here explicitly avoided for denormal arguments, but the operation may **underflow** for other arguments.

This isn't immediately actionable; but might be something for later. Naturally the discussions here in context of IEEE 754 ([IEEEStd754?](#)) (which the Fortran standard does not fully support) and presumes basic knowledge of floating point numbers Goldberg ([1991](#)) . The clearest explanation for requiring higher precision for some constants in these reduction algorithms is given nicely by the draft standard:

For reduction of an argument given in radians, implementations use one or several approximate value(s) of π (or of a multiple of π), valid to, say, n digits. The division implied in the argument reduction cannot be valid to more than n digits, which implies a maximum absolute angle value for which the reduction yields an accurate reduced angle value.

Benchmarking and Testing

To make the benchmarking of new `lfortran` runtime intrinsic is as painless as the existing test runner framework; I ended up implementing a new framework for these. Currently this takes different compiler backends and a TOML file along with some `Jinja2` templates and makes a `CMake` based project which is built and gives out a grid of points along with the `gfortran` values and `abs` differences. Some things to do there:

- ☐ Plot things automatically
- ☐ Use non-uniform grids
- Other QoL changes

This is really easy to work with:

```
cd src/runtime/pure/benchmarks
nix-shell -p 'python38.withPackages(ps: with ps; [seaborn matplotlib click jinja2 tomli pandas])'
# Or just micromamba install seaborn matplotlib pandas click jinja2 tomli
python gen_test.py --execute --build --compiler "gfortran"
```

The TOML file has simple naming rules:

```
[[trig]] # [name of lfortran_MODULE.f90]
fname = "sin" # Function to generate test for; must be the same as for gfortran
only = "dsin" # if not testing same function
range = 20 # [-range, +range]
stepsize = 0.01
```

As does the template:

```
program {{ fname }}_test
  use, intrinsic :: iso_fortran_env, only: sp => real32, dp => real64
  use :: lfortran_intrinsic_trig, only: {{ fname }}2 => {{ fname }}
  real(dp), parameter :: pi = 3.1415926535897932384626433832795_dp
  real(dp) :: ir, gf, lf
  print*, "Compiled with {{ compiler }}"
  print*, "x", "gfortran_{{fname}}(x)", "&
    "lfortran_{{fname}}(x)", "abs_error"
  ir = -{{ range }}
  do while (ir<={{ range }})
    gf = {{ fname }}(ir)
    lf = {{ fname }}2(ir)
    print*, ir, gf, lf, abs(gf-lf)
    ir = ir + {{ stepsize }}
  end do
end program
```

Conclusions

Rather than move forward with the closing comments; and going into more detail about the manner in which further `sollya` based polynomial approximations are being looked into; I will end this post here; as a trailer and invitation to the closing report. This has been the last of the weekly updates. This doesn't mean anything will change other than the format. I quite like keeping track of the work I do in this manner; keeps me honest and not complacent [^fn:3]. For those of you following along; this has been a fun ride; for those who come to this in the future; feel free to follow along from the beginning and drop comments below.

Reading list

Runtime library design, argument reduction, and the floating-point texts behind them, collected as a citation list: [7 references](#). The collection exports to BibTeX or any CSL style, so it can go straight into a bibliography.

References

Brisebarre, Nicolas, David Defour, Peter Kornerup, and Jean-Michel Muller. 2005. "A New Range-Reduction Algorithm." *IEEE TRANSACTIONS ON COMPUTERS* 54 (3): 9.

Defour, P David, Peter Kornerup, Jean-Michel Muller, and Nathalie Revol. n.d. "A New Range Reduction Algorithm," 13.

Goldberg, David. 1991. "What Every Computer Scientist Should Know about Floating-Point Arithmetic." *ACM Computing Surveys* 23 (1): 5--48. <<https://doi.org/10.1145/103162.103163>>.

Muller, Jean-Michel. 2016. *Elementary Functions*. Boston, MA: Birkhäuser Boston. <<https://doi.org/10.1007/978-1-4899-7983-4>>.

Muller, Jean-Michel, Nicolas Brunie, Florent de Dinechin, Claude-Pierre Jeannerod, Mioara Joldes, Vincent Lefèvre, Guillaume Melquiond, Nathalie Revol, and Serge Torres. 2018. *Handbook of Floating-Point Arithmetic*. Cham: Springer International Publishing. <<https://doi.org/10.1007/978-3-319-76526-6>>.

Ng, K C. 2006. "Argument Reduction for Huge Arguments: Good to the Last Bit." *Introduction*, 8.

Payne, Mary H., and Robert N. Hanek. 1983. "Radian Reduction for Trigonometric Functions." *ACM SIGNUM Newsletter* 18 (1): 19--24. <<https://doi.org/10.1145/1057600.1057602>>.

post](<https://stackoverflow.com/questions/13099202/awk-script-for-finding-smallest-value-from-column/13100039>) of course

Hubert](<https://berthub.eu/articles/posts/help-my-sin-is-slow-and-my-fpu-is-inaccurate/>)

see the light of day